プログラミング３ 講義資料

第4回 git入門：基本操作

- バージョン管理の考え方
- リポジトリの構造
- init / add / commit / status

目次

- バージョン管理の概念
- git の基本コマンド

バージョン管理の概念

今回の目的と到達目標

今回の目的

- バージョン管理の必要性を理解
- リポジトリの構造を掴む
- init/add/commit を実行できる

到達目標

- [2] git の基本操作

なぜバージョン管理が必要か

- ファイル名に日付を付ける方式の限界（`report_final_最終.md`）
- 変更履歴を **意味ある単位で** 記録したい
- 誰が・いつ・なぜ変えたかを残したい
- 過去の状態に戻せる保険が欲しい
- 複数人での共同編集を安全に行いたい

> git はこれらの課題を解決するための分散型バージョン管理システム

リポジトリの3領域

ワーキングツリー ⇒ ステージング ⇒ リポジトリ

領域	内容	操作で移動
ワーキングツリー	編集中のファイル	(編集)
ステージング	コミット候補	`git add`
リポジトリ	確定した履歴	`git commit`

- ステージングの役割: **コミットに含める変更を選別する** 場

git の基本コマンド

基本コマンド

```
git init                # リポジトリを作成
git status              # 現在の状態を確認
git add <file>          # ファイルをステージング
git add .               # 全ての変更をステージング
git commit -m "..."     # コミットを作成
git log                 # 履歴を確認
```

- `git status` は迷ったらまず叩く
- コミットメッセージは **何を/なぜ** を書く

コミット粒度の考え方

- 1コミット = 1つの論理的な変更
- 「タイポ修正」と「新機能追加」を同じコミットに混ぜない
- メッセージ例:
  - ❌ `update` / `修正`
  - ✅ `fix: 材料表の分量誤りを修正`
  - ✅ `feat: 手順書にチェックリストを追加`

最初のコミットまでの流れ

Tips

1. `mkdir myproject && cd myproject`
2. `git init`
3. ファイル作成・編集
4. `git status` で変更を確認
5. `git add <file>`
6. `git commit -m "メッセージ"`
7. `git log` で履歴確認

今回のまとめ

- git はワーキングツリー / ステージング / リポジトリの3領域で履歴を管理する
- 基本操作: `init` → 編集 → `add` → `commit` → `log`
- コミットは意味ある単位で、メッセージは「何を/なぜ」

今回カバーした MCC 項目

- V-D-1 プログラミング

次回予告

- 第5回: ブランチとマージ — 並行開発とコンフリクト解消